System Requirements Specification

SRS (Student Planner)

Desktop Productivity Application

Software Engineering

June 7, 2026

1. Introduction

1.1 Project Goal

The goal of the project is to implement the Student Planner application, which is designed to optimize student time management, monitor ongoing learning progress, and keep a record of key academic deadlines. The system provides a number of views presenting information in a clear and intuitive way, allowing for the effective organization of duties.

1.2 Document Conventions

Not applicable.

1.3 Document Audience and Reading Guidelines

This documentation is a substantive compendium of knowledge about the Student Planner system, dedicated to both end users and the development team. Its purpose is to present the functional assumptions and system architecture in detail, allowing for a full understanding of the business logic and technical aspects of the implemented solution.

The document contains a synthetic overview of the project goals, a comprehensive description of system modules, and user interface specifications. This documentation serves as a reference point in the process of further maintenance and development of the application. We recommend paying special attention to chapters 2.2 and 3.1, which contain key design assumptions and functionality definitions that form the foundation of the entire system.

1.4 Product Scope

The Student Planner system is a centralized "management center" for academic processes in the form of a lightweight and efficient desktop application. The application integrates the most important areas of student life organization, providing automation in monitoring deadlines and limits. Thanks to this, the user can focus on the learning process.

A key architectural assumption is to ensure independence from a network connection, which eliminates the risk of data unavailability in case of university server crashes. The use of a local SQLite database allows for offline work, guaranteeing immediate access to resources. This solution ensures a high level of privacy, as all user data is stored exclusively locally.

2. General Description

2.1 Product Perspective

The Student Planner application was developed as a proprietary solution, created from scratch as part of the current development process.

2.2 Product Functions

- Course management: declaration and storage of course data; modification of parameters.
- Calendar: preview of the class schedule and events; handling class notes.
- Progress tracking: monitoring points (from activities and quizzes) along with maximum values and tracking absences.
- Time functions: countdown to the exam session.
- Configuration: changing the interface language (Polish/English).
- Event management: registration, preview, and deletion of events (exams, projects).

- Notifications: notification center informing about upcoming events.

2.3 User Characteristics

The target group consists of university students who need a tool to support the organization of the semester. The application is an alternative solution to university dean's office systems, putting an emphasis on high ergonomics and data readability.

2.4 Runtime Environment

The lightweight architecture of the solution ensures low resource requirements, which translates into smooth operation on various hardware configurations. The standard way to run the application is from source code, using a Python virtual environment and the libraries listed in requirements.txt. An optional Windows executable can be built with PyInstaller (scripts/build.bat). That build is Windows-only and is not required to use the application on other platforms.

2.5 Design Constraints

The visual layer of the application was created using the Kivy framework, which allowed for the efficient implementation of a multi-platform user interface. The application runs on Windows, macOS, and Linux. On Linux, Kivy requires system SDL2 packages. When running the application on macOS, there may be marginal differences in the scaling of selected visual components due to different font rendering and gesture mechanisms in this environment. These discrepancies are purely cosmetic and do not affect the functionality or stability of the system.

The project team is aware of these characteristics and plans to implement a dedicated build in the next phases of product development, which will ensure full visual consistency with macOS design guidelines.

2.6 User Documentation

Navigation: System modules are accessible through the navigation panel (left side of the window). Language switching and notifications are handled from the top right corner.

Calendar: Time navigation is done using dedicated buttons. Selecting a course tile allows for note management.

Adding courses/events: The process is based on filling out forms and confirming the entered data.

Management: Editing and deleting entries is done by interacting with tiles (using function buttons).

Progress Tracker: Modifying the score is done incrementally/decrementally using the "+" and "-" buttons.

2.7 Assumptions and Dependencies

The Student Planner application does not require external IT systems to function properly.

3. External Interfaces

3.1 User Interface (UI)

The user interface was designed with maximum readability and work ergonomics in mind, based on a clear panel layout that segregates system functionalities into logical blocks:

Navigation panel (sidebar): A constant interface element providing quick access to key application modules, such as Calendar, My Subjects, Add Subject, Exams and Tests, Rules Repository, and Progress Tracker. A countdown timer showing the time remaining to the exam session is integrated at the bottom of the panel, allowing for constant monitoring of upcoming deadlines.

Central area (workspace): A dynamic space designed to present the currently selected module. A flexible view system is used here, which ensures readability regardless of the amount of displayed data.

Functional components: Data management is done through dedicated input forms, clear record lists, and modal windows (pop-ups). This solution allows for effective interaction with the system without the need to reload the main view, which significantly improves the comfort of everyday use.

3.2 Hardware Interfaces

The application has been optimized for compatibility with a wide range of PC and laptop devices meeting minimum system requirements. The system interface was designed in a scalable way, which ensures proper display of components on monitors with different resolutions and screen ratios. Thanks to low system resource requirements, Student Planner runs stably on both older units and modern workstations, while ensuring smooth response to standard pointing devices such as a mouse or trackpad.

4. System Features

List of functionalities:

- Course management (add, edit).
- Calendar support (weekly view, notes, events).
- Session countdown module.
- Language configuration.
- Notification module.
- Event management (add, remove).
- Progress monitoring (Progress Tracker).
- Grading rules repository.

5. Non-functional Requirements

5.1 Performance

The system was designed based on an offline-first architecture, which eliminates delays resulting from network communication. The application is characterized by minimal CPU and RAM requirements, ensuring high interface responsiveness (input event response time below the user-noticeable threshold) even on devices with limited performance.

5.2 Security

Data security is based on local isolation. Due to the lack of external network connections, the risk of unauthorized access to information from the outside has been minimized. Data integrity is ensured by the transactional nature of database operations (using the atomic commits mechanism), which protects records from corruption in the event of a sudden application shutdown.

5.3 System Quality Attributes

Usability: The interface design is based on cognitive ergonomics principles, guaranteeing intuitive navigation and quick learning of the system by a new user.

Reliability: The key requirement is error-free CRUD (Create, Read, Update, Delete) operations. Every change in the database is verified, ensuring full consistency of stored information.

Clarity: The data presentation logic has been optimized to maximize readability while maintaining an aesthetic and consistent graphic design.

5.4 User Role Model

The system was designed based on a Single-User role model, which means that the person using the application has full, unrestricted access to all modules and data stored in the local database. Such an architecture simplifies access management and provides the user with full sovereignty over the collected academic information.

6. Other Requirements

A detailed description of business requirements, user requirements, and functional requirements is available in the document containing the project's analytical documentation.

7. Appendices

- Diagrams (activity, use case, class).
- Class description.
- Automated tests documentation.
- Original and actual Gantt.
- Analytical project documentation.